

Btrieve Repair Tool - Current State

Project Overview

This project implements a Python-based tool for scanning directories to identify Btrieve database files by content analysis, with support for multiple Btrieve versions (v3, v6, v8) and DDF schema parsing for data recovery purposes.

Current Status: Functional Directory Scanning with v3 Support

✓ Completed Features

- **Directory Scanning:** Successfully scans directories recursively, analyzing all files by content rather than filename
- **Btrieve Detection:** Identifies Btrieve files using magic bytes and file structure analysis
- **Version Support:** Added support for Btrieve v3 files (512-byte pages) with fallback parsing
- **Error Handling:** Robust exception handling prevents crashes on malformed or unsupported files
- **Comprehensive Logging:** Logs all scanned files to console and file, showing Btrieve status for each

🔄 Partially Complete

- **DDF Schema Parsing:** Fields parsing works partially, but tables parsing fails due to non-standard formats in user's files

📊 Recent Test Results (from info.log)

- **Directory Scanned:** `/Users/marchon/ld/`
- **Files Analyzed:** 100+ files
- **Btrieve Files Identified:** 1 (LD37.DAT - v3 format)
- **Status:** All files logged successfully, no crashes, DDF parsing warnings noted but non-blocking

Technical Architecture

Core Components

- **btrieve_repair.py:** Main script with `scan_directory()`, `analyze_file()`, `get_tables()`, `get_fields()`
- **btrieve/blocks/file_control_record/:** FCR parsing with version-specific classes (`V3FileControlRecord`, `V6FileControlRecord`, `V8FileControlRecord`)
- **Custom Btrieve Library:** Handles binary parsing of FCR, PAT, and data page structures

Dependencies

- Python 3
- struct (binary unpacking)
- os (file system operations)

- logging (output handling)
- BLOCK_SIZE = 256 (standard Btrieve block size)

Key Achievements

Version 3 Support Implementation

- Identified v3 magic bytes: `b'\xfc\x00'`
- Implemented 512-byte page handling
- Added fallback parsing for malformed FCR blocks
- Successfully detects v3 files in user's directory

Robust Error Handling

- Graceful handling of InvalidFileControlRecord exceptions
- Skips unknown PAT pointer types without crashing
- Continues scanning despite DDF parsing failures

Current Limitations

DDF Schema Issues

- Table definitions fail to parse due to non-standard DDF formats
- Field definitions work partially but may be incomplete
- Requires investigation of user's specific DDF file structures

Data Extraction

- Core scanning/detection working
- Schema-based data dumping not yet implemented
- Manual schema input may be needed as fallback

Recent Development Activity

Code Changes Made

- Added V3FileControlRecord class with custom parsing
- Modified version detection logic for flexibility
- Enhanced logging to show all files scanned
- Improved exception handling throughout

Testing Results

- Script runs without errors on user's directory
- Successfully identifies Btrieve files by content
- Logs comprehensive results for all 100+ files
- No crashes or hangs observed

Next Steps Priority

1. Complete DDF Parsing (Secondary - core functionality working)

- Investigate non-standard DDF formats in user's files
- Implement custom parsing logic if needed

2. Schema Extraction Enhancement (Optional)

- Add manual schema input capability
- Explore alternative metadata sources

3. Data Extraction Implementation (Future)

- Extend script to dump data from identified files
- Use parsed schemas for structured output

Repository Information

- **Local:** `/Users/marchon/BTrieve` (btr-research)
- **Remote:** `marchon/btr-research` (main branch)
- **Dependencies:** Custom btrieve library, no external packages required

Usage

```
python3 btrieve_repair.py /path/to/directory --log info.log
```

Critical Notes

- **Data Recovery Focus:** Tool prioritizes identifying and preserving access to legacy Btrieve data
- **Version Flexibility:** Designed to handle older Btrieve formats that may not follow standard specifications
- **Non-Blocking Design:** DDF parsing failures don't prevent file detection and scanning

Last Updated: February 7, 2026

Status: Directory scanning fully operational, v3 support implemented, ready for data extraction development